

HW2

It maybe takes two?

20223085 배상혁

20220611 이규원

20223104 양준영

It maybe takes two?

이름	배상혁 20223085	이규원 20220611	양준영 20223104
업무 경험	게임소프트웨어에서 Unity, GitHub를 사용한 애자일 방식으로 4인 프로젝트 완성 후 배포한 경험	게임 소프트웨어에서 팀장을 맡아 애자일 및 칸반 형식을 사용한 4인 팀 게임 프로젝트 경험	소프트웨어프로젝트2에서 4인 팀 프로젝트로 GitHub, Pygame을 이용한 게임 개발 경험
강점	어떻게든 문제를 해결해내는 끈기	한번 맡은 일은 확실하게 처리하고 끝낸다는 책임감	문제 해결을 위해 자료를 찾고 선별하는 능력
역할	플레이어 구현 및 UI, 씬 관리	게임에서의 멀티 구현 및 전반적인 서버 담당	스테이지 기믹 및 상호작용 가능한 물체 구현

Vision

- Unity & Photon: 안정적이고 꿈김 없는 2인 멀티플레이 환경 구축
- Co-op Puzzle: 유기적인 협동이 필요한 창의적인 퍼즐 레벨 디자인

Scope

- **핵심 시스템 설계 및 구현**

- 캐릭터 컨트롤: 3D 기반 이동, 점프 및 물리 기반 조작
- 퍼즐 기믹: 오브젝트 상호작용(잡기, 밀기) 및 트리거 시스템
- UI/UX: 멀티플레이 로비, 서버 접속 및 인게임 인터페이스

- **멀티플레이어 환경 (2인 협동)**

- Photon 엔진: 실시간 동기화(위치, 애니메이션, 오브젝트 상태)
- 협동 메커니즘: 2인 협력이 필수적인 퍼즐 로직 구현
- 게임 흐름: [접속 → 매칭 → 스테이지 진행 → 클리어] 프로세스 구축

Use Case: 플레이어, UI

담당: 배상혁

Use Case 1: 플레이어 이동 및 시점 조작 (1/2)

Use Case Name	플레이어 이동 및 시점 조작
Scenario	플레이어가 이동 입력과 마우스 시점 입력으로 캐릭터를 조작함
Triggering Event	로컬 입력 권한이 있는 플레이어가 Move, Look, Sprint, Jump 입력을 수행함
Brief Description	PlayerInputHandler 가 입력을 PlayerNetworkInput 으로 수집하고, PlayerMovement 가 카메라 yaw 기준 이 동, 회전, 점프, 스프린트 상태를 네트워크 틱에서 반영함
Actors	플레이어
Related Use Cases	Extend: ESC 설정 메뉴 열기
Stakeholders	플레이어, 게임 시스템
Preconditions	- 플레이어 오브젝트가 스폰되어 입력 권한을 보유해야 함 - PlayerInput , NetworkCharacterController , 카메라가 플레이어 프리팹에 연결되어 있어야 함
Post Conditions	- 플레이어 위치, 회전, 이동 입력, 점프/낙하/스프린트 상태가 갱신됨 - PlayerAnimatorController 가 이동 및 공중 상태 애니메이션 파라미터를 반영함

Use Case 1: 플레이어 이동 및 시점 조작 (2/2)

Actor (플레이어)

1. 이동 및 시점 입력

WASD/스틱으로 이동 방향을 입력하고, 마우스/스틱으로 카메라 yaw와 pitch를 조작함.

2. 달리기와 점프 입력

전방 이동 중 Sprint를 누르면 스프린트 상태가 되고, 지면 위에서 Jump를 누르면 점프가 실행됨.

System (게임 시스템)

3. 입력 수집 및 네트워크 전달

로컬 입력 권한이 있는 클라이언트만 입력을 읽고, 이동 벡터와 카메라 각도 및 버튼 입력을 `PlayerNetworkInput`에 담음.

4. 이동 적용 및 애니메이션 반영

네트워크 틱에서 카메라 방향 기준 이동, 회전, 점프 속도를 적용하고 이동/낙하/스프린트 애니메이션 값을 갱신함.

Exception Conditions

- 1. **입력 차단 상태:** `IsGameplayInputBlocked`가 참이면 이동 벡터와 버튼 입력을 비우고 카메라 각도만 유지함.
- 2. **무입력 상태:** 이동 입력이 없으면 `MoveInput`은 0으로 유지되고 캐릭터는 이동하지 않음.
- 3. **충돌 상태:** 벽, 오브젝트, 무거운 퍼즐 오브젝트와 충돌하면 CharacterController와 push probe 정책에 따라 이동 성분이 제한됨.

Use Case 2: 물리 오브젝트 잡기/놓기/던지기 (1/2)

Use Case Name	물리 오브젝트 잡기/놓기/던지기
Scenario	플레이어가 카메라가 바라보는 물리 오브젝트를 들고 이동하거나 놓고, 조준 방향으로 던짐
Triggering Event	Grab, Throw 입력 또는 Grab 재입력으로 놓기 동작을 수행함
Brief Description	PlayerGrabHandler 가 카메라 raycast로 GrabbableObject 를 찾고, StateAuthority 또는 RPC를 통해 잡기/놓기/던지기를 요청함
Actors	플레이어
Related Use Cases	Include: 플레이어 이동 및 시점 조작
Stakeholders	플레이어, 게임 시스템
Preconditions	• 대상이 GrabbableObject 이며 카메라 raycast와 사거리 검증을 통과해야 함 • 플레이어가 이미 다른 오브젝트를 들고 있지 않아야 함
Post Conditions	• 잡은 오브젝트는 플레이어의 hold point를 따라 이동함 • 놓기 또는 던지기 후 오브젝트의 물리 상태와 충돌 상태가 복구됨

Use Case 2: 물리 오브젝트 잡기/놓기/던지기 (2/2)

Actor (플레이어)

1. 대상 조준 및 잡기

카메라 중앙의 대상 오브젝트를 바라본 상태에서 Grab 입력을 눌러 오브젝트를 잡음.

3. 놓기 또는 던지기

다시 Grab을 누르면 오브젝트를 놓고, Throw를 누르면 현재 조준 방향으로 오브젝트를 던짐.

System (게임 시스템)

2. 대상 검증 및 보유 상태 설정

자기 콜라이더를 제외한 raycast hit를 거리순으로 검사하고, `grabRange + serverValidationPadding` 안의 대상만 잡기 처리함.

4. 물리 상태 전환

들고 있는 동안 rigidbody, gravity, collision을 비활성화하고 hold point에 위치시키며, 해제 시 release overlap 검사를 통과한 위치에서 물리를 복구함.

Exception Conditions

- 1. 대상 없음: raycast가 잡을 수 없는 물체나 자기 콜라이더에만 닿으면 잡기 요청은 무시됨.
- 2. 사거리 초과: 대상의 collider closest point가 허용 사거리 밖이면 잡기 또는 상호작용이 실행되지 않음.
- 3. 해제 위치 충돌: 놓기/던지기 위치가 player body mask와 겹치면 해당 해제 동작은 적용되지 않음.

Use Case 3: 무거운 오브젝트 협동 밀기 (1/2)

Use Case Name	무거운 오브젝트 협동 밀기
Scenario	두 플레이어가 같은 방향으로 무거운 퍼즐 오브젝트를 밀어 제한 범위 안에서 이동시킴
Triggering Event	플레이어가 이동 중 무거운 오브젝트 방향으로 접촉하거나 push probe 범위 안에서 이동 입력을 유지함
Brief Description	PlayerMovement 가 이동 방향의 IPushable 대상에 push force를 전달하고, HeavyObject 는 플레이어별 push sample을 기록해 requiredPushers 이상이 같은 방향일 때 scripted movement를 수행함
Actors	플레이어
Related Use Cases	Include: 플레이어 이동 및 시점 조작
Stakeholders	플레이어, 게임 시스템
Preconditions	- 대상 오브젝트가 HeavyObject 이며 heavy puzzle layer로 설정되어 있어야 함 - 필요한 플레이어 수가 같은 방향으로 짧은 시간 안에 밀어야 함
Post Conditions	- 조건 충족 시 무거운 오브젝트가 방향별로 이동함 - 설정된 이동 범위가 있으면 x/z 좌표가 해당 범위 안으로 제한됨

Use Case 3: 무거운 오브젝트 협동 밀기 (2/2)

Actor (플레이어)

1. 오브젝트 방향으로 이동

플레이어가 무거운 오브젝트를 향해 이동하여 push probe 또는 CharacterController 충돌 판정을 발생시킴.

2. 협동 방향 유지

두 플레이어가 같은 방향으로 계속 밀어 `requiredPushers` 조건을 만족시킴.

System (게임 시스템)

3. push sample 기록

StateAuthority가 플레이어별 push 방향과 시간을 기록하고, 최근 입력 중 같은 방향의 개수를 계산함.

4. scripted movement 적용

조건이 충족되면 일정 시간 동안 오브젝트를 해당 방향으로 이동시키고, 설정된 min/max corner 범위로 위치를 제한함.

Exception Conditions

- 1. 인원 부족: 같은 방향의 active push 수가 `requiredPushers`보다 적으면 오브젝트는 이동하지 않음.
- 2. 방향 충돌: 같은 수의 서로 다른 방향 입력이 동시에 감지되면 이동 방향을 확정하지 않음.
- 3. 범위 설정 누락: 이동 범위 corner 중 하나만 지정된 경우 경고를 출력하고 scripted movement를 적용하지 않음.

Use Case 4: ESC 설정 메뉴 열기 (1/2)

Use Case Name	ESC 설정 메뉴 열기
Scenario	플레이어가 스테이지 진행 중 ESC 키로 설정 메뉴를 열고 닫거나 게임 나가기를 선택함
Triggering Event	MainStageScene 에서 플레이어가 ESC 키를 누름
Brief Description	StageSettingsMenu 가 ESC 입력을 감지해 설정 오버레이를 토글하고, 메뉴가 열려 있는 동안 게임플레이 입력과 크로스헤어를 차단한 뒤 커서를 UI 조작 상태로 전환함
Actors	플레이어
Related Use Cases	Include: 플레이어 이동 및 시점 조작
Stakeholders	플레이어, 게임 시스템
Preconditions	• 현재 씬이 MainStageScene 이어야 함 • StageSettingsMenu 가 씬 로드 시 생성되고 StageSettingsCanvas 가 준비되어 있어야 함
Post Conditions	• 메뉴가 열리면 Settings 오버레이와 Close/Leave Game 버튼이 표시됨 • 메뉴가 닫히면 게임플레이 입력, 커서 상태, 크로스헤어 표시 상태가 이전 상태로 복구됨

Use Case 4: ESC 설정 메뉴 열기 (2/2)

Actor (플레이어)

1. 메뉴 열기

스테이지 진행 중 ESC 키를 눌러 Settings 오버레이를 호출함.

3. 메뉴 닫기 또는 나가기

Close 버튼이나 ESC 키로 메뉴를 닫거나, Leave Game 버튼으로 세션을 떠나 로비로 이동함.

System (게임 시스템)

2. 입력 차단 및 UI 표시

메뉴를 열 때 `PlayerInputHandler.IsGameplayInputBlocked``를 true로 설정하고 로컬 `PlayerInput``과 크로스헤어를 비활성화한 뒤 커서를 표시함.

4. 상태 복구 또는 씬 전환

Close 시 이전 입력/커서/크로스헤어 상태를 복구하고, Leave Game 시 세션을 떠난 뒤 로비 씬으로 이동함.

Exception Conditions

- 1. 메인 스테이지 아님: 현재 씬이 `MainStageScene``이 아니면 `StageSettingsMenu`` 오브젝트를 생성하지 않거나 즉시 제거함.
- 2. 나가기 처리 중: `isLeaving`` 상태에서는 ESC 입력과 Close 처리를 무시하고 중복 나가기 요청을 막음.
- 3. 매니저 누락: `SceneFlowManager``가 없으면 로비 씬 직접 로드 또는 빌드 인덱스 0 로드로 fallback 처리함.

Non-Functional Requirements

Player

Use Case Name	NFR 내역 (Non-Functional Requirements)	Quality	Quality Attributes
플레이어 이동 및 시점 조작	입력 응답성: 로컬 입력 권한을 가진 플레이어만 입력을 수집하고, 입력은 다음 네트워크 틱에 반영되어야 함.	Performance Efficiency (성능 효율성)	<code>OnInput</code> 에서 입력을 설정하고, 버튼 입력은 전송 후 즉시 버퍼를 비움.
플레이어 이동 및 시점 조작	이동 상태 일관성: 이동 벡터, 카메라 각도, 스프린트, 점프, 낙하 상태가 일관되게 반영되어야 함.	Functional Suitability (기능 적합성)	<code>PlayerMovement</code> 의 Networked 값과 애니메이션 파라미터를 같은 상태 기준으로 갱신함.
물리 오브젝트 잡기/놓기/던지기	상호작용 정확성: 대상은 카메라 raycast, 자기 collider 제외, 사거리 검증을 모두 통과해야 함.	Functional Suitability (기능 적합성)	<code>grabRange + serverValidationPadding</code> 안의 유효 대상만 RPC/StateAuthority로 처리함.
물리 오브젝트 잡기/놓기/던지기	물리 상태 안정성: 들고 있는 동안 충돌/중력을 비활성화하고, 해제 시 겹침 검사를 통과해야 함.	Reliability (신뢰성)	held 모드로 물리를 전환하고, release overlap 실패 시 drop/throw를 거부함.

Use Case Name	NFR 내역 (Non-Functional Requirements)	Quality	Quality Attributes
무거운 오브젝트 협동 밀기	협동 판정 정확성: 요구 인원 이상이 같은 방향으로 최근 push sample을 남긴 경우에만 이동해야 함.	Functional Suitability (기능 적합성)	<code>requiredPushers</code> 이상이고 방향 동률이 없을 때만 scripted movement를 활성화함.
무거운 오브젝트 협동 밀기	이동 범위 안정성: scripted movement 중 오브젝트가 지정된 x/z 범위를 벗어나지 않아야 함.	Reliability (신뢰성)	min/max corner가 있으면 좌표를 clamp하고, 범위 설정이 불완전하면 이동하지 않음.
ESC 설정 메뉴 열기	메뉴 조작 사용성: ESC 키로 설정 메뉴를 열고 닫으며, 닫기와 게임 나가기 선택지를 제공해야 함.	Usability (사용성)	<code>StageSettingsMenu</code> 가 오버레이와 버튼을 생성하고 ESC로 open/close를 토글함.
ESC 설정 메뉴 열기	입력 차단 일관성: 메뉴가 열린 동안 조작 입력과 크로스헤어를 차단하고, 닫으면 이전 상태로 복구해야 함.	Functional Suitability (기능 적합성)	<code>IsGameplayInputBlocked</code> , <code>PlayerInput</code> , <code>showCrosshair</code> , Cursor 상태를 저장/복구함.

Use Case: 서버

담당: 이규원

Use Case 1: 방 생성하기 (1/2)

Use Case Name	방 생성하기 (Create Room)
Scenario	사용자가 게임서버에 접속하여 새로운 방을 생성함
Triggering Event	사용자가 '방 생성하기' 버튼을 누름
Brief Description	시스템은 플레이어의 요청을 받아 새로운 Photon Room을 생성하며, 이때 방 이름, 최대 인원, 공개 여부 등의 속성을 설정함
Actors	플레이어
Related Use Cases	Include: 방 참가하기 Extend: 게임 시작하기
Stakeholders	플레이어, 같은 방에 입장할 다른 플레이어들, 게임 시스템
Preconditions	• 플레이어가 서버에 연결되어 있어야 함 • 방 생성 화면 또는 로비 상태에 있어야 함 • 방 생성에 필요한 정보(방 이름, 최대 인원 등)가 입력되어야 함
Post Conditions	• 새로운 방이 생성되고, 방을 생성한 플레이어가 즉시 입장됨 • 방 정보(속성)가 설정됨 • 공개 방이면 타인의 검색/입장이 가능하며, 비공개 방이면 이름을 아는 사용자만 입장 가능함

Use Case 1: 방 생성하기 (2/2)

Actor (플레이어)

1. 메뉴 선택 및 정보 입력

방 생성 메뉴를 선택한 뒤, 방 이름, 최대 인원 등의 속성 정보를 입력함.

2. 생성 요청

생성 버튼을 눌러 서버에 방 생성 요청을 전송함.

System (Photon 서버)

3. 유효성 검사 및 방 생성

입력값의 유효성을 확인한 후, 새로운 방을 생성하고 방 정보를 서버에 저장함.

4. 플레이어 입장 및 UI 갱신

방 생성이 완료되면 플레이어를 해당 방에 입장시키고 대기실 화면을 표시함.

Exception Conditions

- 1. **연결 오류**: 서버 연결이 끊어진 상태일 경우 방 생성 실패 처리 및 알림.
- 2. **입력값 오류**: 중복되거나 유효하지 않은 방 이름, 또는 입력값이 누락된 경우 생성 거부.
- 3. **서버 요청 실패**: 네트워크 문제 등으로 Photon 서버 생성 요청에 실패한 경우 에러 메시지 출력.
- 4. **잘못된 속성**: 허용되지 않은 방 속성(예: 비정상적인 최대 인원 수) 설정 시 생성 제한.

Use Case 2: 방 참가하기 (1/2)

Use Case Name	방 참가하기 (Join Room)
Scenario	플레이어가 기존에 생성된 방에 입장하기 위해 참가 요청을 보냄
Triggering Event	플레이어가 방 목록에서 특정 방을 선택하거나, 방 이름을 입력하여 참가를 요청함
Brief Description	시스템은 플레이어의 방 참가 요청을 받아 해당 방의 존재 여부, 입장 가능 여부, 최대 인원 등을 기준으로 참가를 처리함
Actors	플레이어
Related Use Cases	Include: 서버 접속하기, 방 검색하기 Extend: 방 생성하기
Stakeholders	플레이어, 같은 방의 기존 플레이어들, 게임 시스템
Preconditions	• 플레이어가 Photon 서버에 연결되어 있어야 함 • 참가하려는 방이 존재하거나 조건에 맞는 방이 있어야 함 • 방이 닫혀 있지 않고 최대 인원에 도달하지 않아야 함
Post Conditions	• 플레이어가 해당 방에 입장하며 방 인원 수가 갱신됨 • 같은 방의 기존 플레이어들에게 새 참가자의 입장이 동기화됨

Use Case 2: 방 참가하기 (2/2)

Actor (플레이어)

1. 방 탐색 및 선택

방 목록을 확인하고 참가할 방을 선택하거나 방 이름을 직접 입력함.

2. 참가 요청

참가 버튼을 눌러 서버에 해당 방으로의 입장을 요청함.

System (Photon 서버)

3. 방 상태 검증

요청받은 방의 존재 여부, 현재 인원 수 및 입장 가능 상태(비밀 번호 등)를 확인함.

4. 입장 처리 및 갱신

검증 통과 시 플레이어를 방에 입장시키고, 참가자 목록을 갱신하여 대기실 화면을 표시함.

Exception Conditions

- 1. **방 존재 안 함:** 방이 이미 사라졌거나 존재하지 않는 경우 참가 실패 처리 및 알림.
- 2. **인원 초과:** 방이 이미 최대 인원수에 도달하여 가득 찬 경우 입장 거부 및 에러 메시지 출력.
- 3. **중복 입장:** 동일한 UserID로 같은 방에 이미 입장이 있는 상태에서 중복 참가를 요청할 경우 처리 거부.

Use Case 3: 채팅하기 (1/2)

Use Case Name	채팅하기 (Chatting)
Scenario	같은 게임 방에 입장한 플레이어들이 대기실 또는 플레이 중 텍스트 메시지를 주고받음
Triggering Event	플레이어가 채팅창에 메시지를 입력하고 전송(Enter) 키를 누름
Brief Description	시스템은 플레이어가 입력한 메시지를 RPC(Remote Procedure Call)를 통해 같은 방의 다른 플레이어들에게 전달하고 UI에 표시함
Actors	플레이어
Related Use Cases	Include: 서버 접속하기, 방 생성하기, 방 참가하기
Stakeholders	플레이어, 같은 방의 다른 플레이어, 게임 시스템
Preconditions	• 플레이어가 서버에 연결되어 있고, 같은 방에 입장한 상태여야 함 • 채팅 UI가 활성화되어 있어야 함 • 채팅 처리 오브젝트에 네트워크 컴포넌트(PhotonView)가 연결되어 있어야 함
Post Conditions	• 전송된 메시지가 방 안의 모든 플레이어에게 전달됨 • 발신자 정보와 함께 각 플레이어의 채팅창 화면에 메시지가 표시됨

Use Case 3: 채팅하기 (2/2)

Actor (플레이어)

1. 메시지 작성

채팅 입력창을 활성화하고 전송할 텍스트 메시지를 작성함.

2. 전송 요청

Enter 키 또는 전송 버튼을 눌러 메시지 발송을 서버에 요청함.

System (Photon 서버)

3. 유효성 검증 및 브로드캐스트

입력된 메시지가 비어 있는지 확인한 후, RPC를 통해 방 안의 다른 플레이어들에게 메시지를 전파함.

4. UI 갱신

각 클라이언트의 채팅 UI에 발신자 정보와 함께 수신된 메시지를 출력함.

Exception Conditions

- 1. **연결 오류**: 서버 연결이 끊어진 경우 메시지 전송이 실패하며 에러 처리됨.
- 2. **방 미입장 상태**: 로비 등 방에 입장하지 않은 상태에서는 채팅 기능을 사용할 수 없음.
- 3. **빈 메시지 전송**: 입력된 텍스트가 비어 있거나 공백만 있는 경우, 시스템이 전송 요청을 무시함.

Use Case 4: 스테이지 시작하기 (1/2)

Use Case Name	스테이지 시작하기 (Start Stage)
Scenario	같은 방에 있는 플레이어들의 준비 상태가 충족되면 시스템이 게임 시작 여부를 방 전체에 반영함
Triggering Event	게임 시작 조건이 충족되거나, 플레이어(방장)가 시작 요청을 보냄
Brief Description	시스템은 게임 시작 여부, 시작 시간 등의 정보를 룸 속성(Room Properties)으로 갱신하고, 같은 방의 모든 플레이어에게 이를 동기화함
Actors	플레이어
Related Use Cases	Include: 방 참가하기, Ready 하기, 방 인원 Ready 체크
Stakeholders	플레이어, 같은 방의 다른 플레이어, 게임 시스템
Preconditions	• 플레이어들이 같은 Photon Room에 입장해 있어야 함 • 모든 플레이어의 게임 시작 조건(Ready 등)이 충족되어야 함 • 시작 상태를 저장할 룸 속성(HashTable 형태)이 정의되어 있어야 함
Post Conditions	• 게임 시작 여부가 룸 전체 상태 속성에 갱신/반영됨 • 필요 시 시작 시간 및 라운드 정보가 함께 저장됨 • 같은 방의 모든 플레이어 화면이 동일한 인게임 상태로 전환됨

Use Case 4: 스테이지 시작하기 (2/2)

Actor (플레이어)

1. 대기 및 준비

같은 방에 입장하여 게임 시작을 위한 준비(Ready) 상태로 대기함.

2. 시작 요청

조건이 충족된 상태에서 플레이어(방장)가 게임 시작 버튼을 누름.

System (Photon 서버)

3. 시작 조건 검증

시작 요청을 수신한 뒤, 현재 방 인원과 모든 플레이어의 준비 상태를 확인함.

4. 룸 속성 갱신 및 전환

시작이 가능하다고 판단되면 룸 속성(시작 시간, 상태)을 갱신하고 모든 플레이어의 화면을 게임 진행 상태로 일제히 전환함.

Exception Conditions

- 1. **인원 부족**: 일부 플레이어가 아직 방에 입장하지 않았거나 최소 시작 인원에 미달한 경우.
- 2. **준비 미완료**: 방 안의 모든 플레이어가 'Ready' 상태가 아닌 경우 시작 요청 거절.
- 3. **속성 갱신 실패**: 네트워크 문제 등으로 Photon 룸 속성(HashTable) 갱신에 실패한 경우 진행 중단.

Non-Functional Requirements

비기능적 요구사항

Use Case Name	NFR 내역 (Non-Functional Requirements)	Quality	Quality Attributes
방 생성하기	방 생성 응답성: 사용자가 방 생성 요청을 보냈을 때, 시스템은 지연 없이 방 생성 결과를 반환하고 대기실 화면으로 전환해야 함.	Performance Efficiency (성능 효율성)	Time Behavior(시간 반응성) 평균 2초 이내에 방 생성 결과가 화면에 반영되고 대기실로 전환되어야 함.
방 참가하기	입장 판정 정확성: 존재하지 않는 방, 최대 인원을 초과한 방, 닫힌 방에 대해서는 참가 요청이 정확하게 거부되어야 함.	Functional Suitability (기능 적합성)	Fault Tolerance(결함 수용성) 동시 요청 상황에서도 최대 인원을 초과한 입장을 허용하지 않아야 함.
채팅하기	메시지 전달 응답성: 플레이어가 전송한 채팅 메시지는 같은 방의 다른 플레이어들에게 짧은 시간 안에 표시되어야 함.	Performance Efficiency (성능 효율성)	Confidentiality(기밀성) 같은 방 외부의 사용자에게 채팅 메시지가 전달되거나 노출되지 않아야 함.
스테이지 시작하기	시작 조건 판정 정확성: 모든 플레이어가 Ready 상태일 때만 게임 시작이 허용되어야 하며, 조건이 충족되지 않으면 시작 요청이 거부되어야 함.	Functional Suitability (기능 적합성)	Interoperability(상호운용성) 모든 클라이언트가 동일한 시작 상태와 시작 시점을 공유해야 함.

Use Case: 오브젝트, 맵

담당: 양준영

Use Case 1: 스테이지 입장 - 맵 생성 (1/2)

Use Case Name	스테이지 입장 - 맵 생성 (Load Stage Map)
Scenario	플레이어가 스테이지를 선택하면 맵이 생성되고 아바타가 배치됨
Triggering Event	플레이어가 스테이지를 선택하여 입장을 시도함
Brief Description	게임 시스템은 플레이어가 선택한 스테이지를 확인하여 해당하는 맵을 로딩하고, 완료 시 플레이어의 아바타를 스폰 위치에 생성함
Actors	플레이어
Related Use Cases	Include: 서버 - 게임 시작하기
Stakeholders	플레이어, 게임 시스템
Preconditions	• 게임이 실행 중이어야 함 • 플레이어들이 게임에 접속해 시작할 준비를 마친 상태여야 함
Post Conditions	• 스테이지의 맵이 로드됨 • 게임에 접속한 플레이어들의 아바타가 맵의 지정된 위치에 놓임

Use Case 1: 스테이지 입장 - 맵 생성 (2/2)

Actor (플레이어)

1. 스테이지 선택

스테이지 목록이 나열된 화면에서 플레이할 스테이지를 찾아 선택함.

System (게임 시스템)

2. 씬 이동 및 맵 로딩

맵을 로딩할 씬으로 이동한 후, 선택된 스테이지를 확인하여 해당하는 맵 데이터를 매니저를 통해 로딩함.

3. 아바타 스폰 및 준비 완료

로딩이 완료되면 플레이어의 아바타를 생성해 맵의 스폰 포인트에 위치시키고, 게임을 시작할 수 있도록 처리함.

Exception Conditions

- 1. **연결 끊김**: 로딩이 진행되는 과정에서 플레이어와의 네트워크 연결이 끊긴다면, 알림 팝업을 띄우고 즉시 로비로 이동 처리함.

Use Case 2: 기믹 수행 - 레버 작동 (1/2)

Use Case Name	기믹 수행 - 레버 작동 (Operate Lever Gimmick)
Scenario	플레이어가 레버 오브젝트와 상호작용해 숨겨진 구조물을 활성화함
Triggering Event	플레이어가 레버 오브젝트와 상호작용을 시도함
Brief Description	플레이어가 레버를 작동시키면, 아바타 조작만으로는 오를 수 없는 절벽에 비계 형태의 구조물이 나타나 이동 가능한 길을 만들어 줌
Actors	플레이어
Related Use Cases	없음
Stakeholders	플레이어, 게임 시스템
Preconditions	• 게임이 실행 중이며 플레이어가 아바타를 조작할 수 있는 상태여야 함 • 오를 수 없는 절벽 지형과 상호작용 가능한 레버가 존재해야 함
Post Conditions	• 레버가 작동 후 상호작용 불가(비활성화) 상태로 전환됨 • 절벽을 오를 수 있는 비계 형태의 구조물이 맵에 생성/등장함

Use Case 2: 기믹 수행 - 레버 작동 (2/2)

Actor (플레이어)

1. 상호작용 시도

절벽 근처에 위치한 레버 오브젝트에 다가가 상호작용 키를 입력함.

System (게임 시스템)

2. 상태 비활성화

레버와의 상호작용 입력을 감지하면 즉시 해당 레버를 상호작용 불가 상태로 변경함.

3. 구조물 활성화

절벽 속이나 땅 아래에 숨어 있던 비계 형태의 지형 구조물을 맵 상에 등장시킴.

Exception Conditions

- 1. 중복 작동 시도: 이미 작동이 완료되어 비활성화된 레버에 다시 상호작용을 시도할 경우, 시스템은 이를 무시하고 아무런 이벤트도 발생시키지 않음.

Use Case 3: 기믹 수행 - 발판 작동 (1/2)

Use Case Name	기믹 수행 - 발판 작동 (Operate PressurePlate Gimmick)
Scenario	플레이어가 발판 오브젝트와 상호작용해서 구조물이 움직이게 함
Triggering Event	두 플레이어가 각각 발판 오브젝트 위에 올라섬
Brief Description	플레이어들이 발판 오브젝트 위에 올라감. 발판이 작동하면 플랫폼 구조물이 움직이기 시작. 발판 위에서 내려오면, 움직이던 플랫폼 구조물은 움직임을 멈춤
Actors	플레이어
Related Use Cases	없음
Stakeholders	플레이어, 게임 시스템
Preconditions	• 게임이 실행 중이며 플레이어가 아바타를 조작할 수 있는 상태여야 함 • 두 개의 발판과 플랫폼 오브젝트가 존재해야 함
Post Conditions	플랫폼 구조물이 지속적으로 움직이기 시작함

Use Case 3: 기믹 수행 - 발판 작동 (2/2)

Actor (플레이어)

1. 최초 상호작용 시도

두 명의 플레이어가 두 개의 발판 오브젝트 위에 각각 올라감

3. 추가 상호작용 시도

두 플레이어 중 최소 한 명이 발판 위에서 내려감

System (게임 시스템)

2. 1차 상태 전환

플랫폼 오브젝트가 지속적으로 움직이기 시작함

4. 2차 상태 전환

플랫폼 오브젝트의 움직임이 멈춤

Exception Conditions

- 1. 발판에 정확히 위치: 발판 오브젝트의 트리거 구역에 정확하게 위치하지 않으면 발판은 반응하지 않음.

Use Case 4: 최종 기믹 해결 및 스테이지 클리어 (1/2)

Use Case Name	최종 기믹 해결 및 스테이지 클리어 (Clear Final Gimmick & Stage)
Scenario	스테이지의 최종 기믹을 수행하고 스테이지 클리어 상태로 진입함
Triggering Event	플레이어들이 스테이지에 설정된 최종 클리어 조건을 달성함
Brief Description	두 명의 플레이어가 각각 두 개의 발판을 밟아 출구를 개방하고, 두 플레이어 모두 해당 출구로 진입하여 스테이지를 클리어함
Actors	Primary: 플레이어 A, 플레이어 B
Related Use Cases	Include: 플레이어 - 스테이지 이동 및 클리어하기
Stakeholders	플레이어, 게임 시스템
Preconditions	• 게임이 실행 중이며 아바타를 조작할 수 있는 상태여야 함 • 클리어 조건에 해당하는 두 개의 발판과 닫혀있는 출구가 존재해야 함
Post Conditions	• 시스템이 스테이지가 클리어된 것을 확인하고 클리어 관련 절차에 돌입함

Use Case 4: 최종 기믹 해결 및 스테이지 클리어 (2/2)

Actor (플레이어 A, B)

1. 발판 활성화 (기믹 수행)

두 명의 플레이어가 각각 맵에 배치된 두 개의 발판 오브젝트 위에 올라섬.

3. 출구 진입

기믹이 풀려 출구 오브젝트가 열리면, 두 플레이어 모두 해당 출구 영역으로 진입함.

System (게임 시스템)

2. 기믹 판정 및 출구 개방

두 개의 발판이 모두 밟혀 활성화된 것을 감지하면, 닫혀있던 출구 오브젝트를 활성화하여 문을 엽니다.

4. 클리어 판정 및 절차 실행

두 플레이어의 출구 진입이 모두 감지되면 스테이지 클리어 조건을 충족시키고 클리어 절차를 실행함.

Exception Conditions

- 1. **연결 끊김:** 스테이지 클리어 관련 절차를 실행하기 이전에 네트워크 연결이 끊긴다면, 실행 절차를 즉시 중단하고 재시도 팝업을 띄우거나 메인 메뉴로 강제 이동시킴.

Non-Functional Requirements

비기능적 요구사항

Use Case Name	NFR 내역 (Non-Functional Requirements)	Quality	Quality Attributes
스테이지 입장 - 맵 생성	스테이지 생성 응답성: 플레이어가 스테이지를 선택하고 게임을 시작하려 할 때 맵 로딩이 신속히 진행되어야 함.	Performance Efficiency (성능 효율성)	Time Behavior: 지정된 시간(약 3초) 안에 스테이지가 로딩되도록 함.
기믹 수행 - 레버 작동	상호작용 정확성: 레버 상호작용 시 지정된 오브젝트들이 정확히 작동하도록 해야 함.	Functional Suitability (기능 적합성)	Functional Correctness: 레버와 지정된 오브젝트를 정확히 연결해 함수가 올바르게 작동하도록 구현.
기믹 수행 - 버튼 작동	기믹 학습성: 복잡한 기믹이라도 사용자가 빠른 시간 내에 이해할 수 있게 설계해야 함.	Interaction Capability (상호작용 능력)	Learnability: 한 화면 안에서 기믹의 실행이 파악되는 등 기믹을 직관적으로 이해 가능하게 설계.
스테이지 최종 기믹 해결 및 스테이지 클리어	클리어 판정 무결성: 최종 기믹을 정상적으로 수행했을 시에만 스테이지 클리어 판정으로 인식해야 함.	Reliability (안정성)	Faultlessness: 지정된 기믹만 클리어 판정으로 연결, 이외 모든 기믹에 대해 클리어 판정으로 연결되지 않도록 전수조사